

FA2 Case Study

Data Analysis Using Machine Learning and Streamlit App Deployment

Dataset: Mobile Price Classification (Kaggle)

Name: Ishan Vinodrao Jirapure

PRN: 125M1H043

Course: Advance Data Science

1. Introduction

When a new phone is launched, its price depends on a combination of specifications such as RAM, battery, camera and screen size. In this case study I used machine learning to predict the price category of a mobile phone from its technical specifications. The task is a multiclass classification problem, because the phone has to be placed into one of four price groups rather than being given an exact price.

The work follows the complete machine learning workflow that we studied in class: choosing a dataset, exploring and cleaning the data, splitting it into training and testing sets, building more than one model, comparing the models using proper metrics, tuning the best model with GridSearchCV, and finally deploying the model as a small web application using Streamlit.

1.1 Objectives

- To understand the structure of the Mobile Price Classification dataset and find which features matter most.
- To train and compare four classification algorithms: Logistic Regression, Decision Tree, Random Forest and SVM.
- To improve the Random Forest model using 5-fold GridSearchCV.
- To deploy the final model as a Streamlit app that predicts the price category from user inputs.

2. Dataset Description

The dataset was downloaded from Kaggle and is stored in a file called train.csv. It contains mobile-phone technical specifications and a target variable called price_range. The target has four classes: 0 for Low Cost, 1 for Medium Cost, 2 for High Cost and 3 for Very High Cost. There are 20 input features, covering battery power, RAM, internal memory, camera specifications, screen dimensions, connectivity and processor information. The dataset has 2000 records, and the four classes are equally represented with 500 phones each, so there is no class imbalance problem.

Table 1 lists the input features and what each of them represents.

Feature	Description
battery_power	Total battery capacity in mAh
blue	Whether the phone has Bluetooth (0 / 1)
clock_speed	Speed at which the microprocessor executes instructions
dual_sim	Whether the phone supports dual SIM (0 / 1)
fc	Front camera megapixels
four_g / three_g	Whether 4G / 3G is supported (0 / 1)
int_memory	Internal memory in GB
m_dep	Mobile depth in cm
mobile_wt	Weight of the phone
n_cores	Number of processor cores
pc	Primary (rear) camera megapixels
px_height / px_width	Pixel resolution height / width
ram	Random access memory in MB

Feature	Description
sc_h / sc_w	Screen height / width in cm
talk_time	Longest time a single battery charge lasts during a call
touch_screen	Whether it is a touch screen phone (0 / 1)
wifi	Whether the phone has Wi-Fi (0 / 1)

Table 1: Input features of the dataset

3. Data Preprocessing and EDA

3.1 Loading and inspecting the data

The notebooks start by loading train.csv with pandas. I first looked at the first few rows using head(), then checked the data types with info(), counted missing values with isnull().sum(), checked for duplicate rows with duplicated().sum(), and generated a statistical summary with describe(). These checks are important because models give unreliable results if the data contains empty cells, repeated records or wrongly typed columns.

All the input columns are numeric (integers or floats), including the binary columns such as blue, wifi and touch_screen which are already stored as 0 and 1. Because of this, no categorical encoding such as one-hot or label encoding was needed.

3.2 Exploratory data analysis

Several visualizations were created to understand the data before modelling:

- **Price-range distribution:** a count plot of the target to confirm that the four classes are balanced.
- **RAM distribution:** a plot of RAM values to see how widely they are spread across phones. RAM varies a lot, from very small to very large values, which makes it a useful feature.
- **Battery power versus RAM:** a plot comparing these two features, coloured by price range, to see whether the price groups separate visually.
- **Correlation heatmap:** a heatmap of the correlation between all the columns. From it, RAM shows the strongest relationship with price_range, while most other features show only a weak relationship on their own.

The main takeaway from the EDA is that RAM is the most important feature for price, and that the features have very different scales (for example, RAM is in thousands while clock_speed is below 3). This second point matters when selecting and preparing models.

3.3 Train-test split and scaling

The features were separated from the target (price_range) and the data was divided using an 80/20 stratified train-test split. This gives 1600 records for training and 400 for testing. Stratification keeps the same proportion of each price class in both sets, so the test results are fair. A fixed random_state was used so that the results can be reproduced.

StandardScaler was applied for Logistic Regression and SVM. These two algorithms are sensitive to the scale of the features: if one feature has very large numbers, it can dominate the others. The scaler was fitted only on the training data and then used to transform the test data, so that no information from the test set leaks into training. Decision Tree and Random Forest do not depend on feature scale, so they were trained on the original values.

4. Machine Learning Models

Four different classification algorithms were used so that I could compare simple and complex approaches. Table 2 gives a short summary and the next paragraphs explain each one in my own words.

Model	Purpose	Uses scaled data?
Logistic Regression	Multiclass classification baseline	Yes
Decision Tree	Tree-based classification	No
Random Forest	Ensemble classification	No
SVM	Margin-based classification	Yes

Table 2: Models used in this study

Logistic Regression. This is a linear model that estimates the probability of each class. For more than two classes it is extended to the multiclass case. I used it as the baseline because it is simple and fast, and any other model should perform at least as well.

Decision Tree. This model splits the data again and again using questions like "is RAM greater than a certain value?" until it reaches a decision. It is easy to understand, but a single deep tree can overfit the training data.

Random Forest. This is an ensemble of many decision trees, each trained on a random part of the data and features. The final answer is decided by voting. Combining many trees reduces overfitting and usually gives better and more stable results than one tree.

Support Vector Machine (SVM). SVM tries to find the boundary that separates the classes with the largest possible margin. It works well on scaled numeric data, which is why StandardScaler was applied before training it.

5. Model Evaluation

Each model was evaluated on the 400 test records using three tools: accuracy, the classification report and the confusion matrix.

- **Accuracy** is the fraction of phones whose price category was predicted correctly.
- **Classification report** shows precision, recall and F1-score for each of the four classes. Precision tells how many of the phones predicted in a class really belong to it, and recall tells how many of the real phones of that class were found.
- **Confusion matrix** shows exactly which classes get confused with each other. In this problem, mistakes are most likely between neighbouring classes, for example Medium Cost and High Cost, rather than between Low Cost and Very High Cost.

The exact values were taken from the executed notebooks and are entered in Table 3. I have copied the scores only after running the models on my own system so that the report matches my actual execution.

6. Hyperparameter Tuning with GridSearchCV

Random Forest was chosen for tuning because it is a strong and flexible model. Instead of guessing the settings by hand, I used GridSearchCV, which tries every combination of the given parameters and measures each one using cross-validation. Cross-validation with 5 folds splits the training data

into five parts, trains on four and validates on the remaining one, and repeats this five times. The average score is therefore more reliable than a single split.

Parameter	Values searched
n_estimators	50, 100
max_depth	None, 10, 20
min_samples_split	2, 5

Table 4: Parameter grid for Random Forest

These values make $2 \times 3 \times 2 = 12$ parameter combinations. With 5-fold cross-validation, GridSearchCV trains $12 \times 5 = 60$ models in total. The notebook prints the best parameters and the best cross-validation accuracy.

Best parameters: [copy from notebook]

Best cross-validation accuracy: [copy from notebook]

Here n_estimators controls the number of trees, max_depth limits how deep each tree can grow (which helps to control overfitting), and min_samples_split sets the minimum number of samples needed to split a node. After the search, the best model was saved to a file called best_mobile_price_model.pkl using joblib/pickle, so that it can be loaded later in the Streamlit app without training it again.

7. Streamlit Application

To make the model usable by someone who does not know programming, I built a simple web application with Streamlit. The app accepts the 20 mobile specifications through simple input widgets such as number inputs, sliders and select boxes. When the user clicks the predict button, the app loads the saved Random Forest model, passes the values to it in the same feature order used during training, and displays the predicted price category (Low, Medium, High or Very High Cost).

7.1 Working of the app

- The saved model best_mobile_price_model.pkl is loaded once when the app starts.
- The user enters the values for battery power, RAM, internal memory, camera, screen size, connectivity and the other features.
- The inputs are arranged into a single row in the correct column order.
- The model predicts the class (0 to 3), which is then converted into a readable label and shown on the screen.

A simplified version of the main idea of app.py is shown below.

```
import streamlit as st
import joblib, numpy as np

model = joblib.load("best_mobile_price_model.pkl")
labels = ["Low Cost", "Medium Cost", "High Cost", "Very High Cost"]

st.title("Mobile Price Prediction")
ram = st.number_input("RAM (MB)", 256, 4000, 2000)
battery = st.number_input("Battery power (mAh)", 500, 2000, 1200)
# ... remaining inputs for all 20 features ...

if st.button("Predict"):
    pred = model.predict(np.array([[battery, ..., ram, ...]]))[0]
    st.success("Predicted category: " + labels[pred])
```

The app can be started from the terminal with the command: `streamlit run app.py`. It then opens in the browser, usually at `http://localhost:8501`.

8. Observations and Limitations

- The dataset is balanced and clean, which makes the classification task easier than many real-world problems.
- RAM is the most influential feature. Features like battery power, pixel resolution and screen size also add useful information when combined.
- Scaling was necessary for Logistic Regression and SVM but not for the tree-based models.
- Tuning with GridSearchCV is useful but takes more time as the grid grows, so only a small and sensible grid was used.
- The data has only 2000 records and one source, so the model may not work equally well on phones from a different market or time period. A larger and more recent dataset would improve it.

For future work, the project could be extended by trying other models such as Gradient Boosting or XGBoost, using feature importance to remove weak features, adding a prediction probability in the app, and hosting the app online using Streamlit Community Cloud so that others can use it.

9. Conclusion

This project demonstrates a complete machine-learning workflow: dataset selection, EDA, preprocessing, train-test split, four classification algorithms, model evaluation, 5-fold GridSearchCV and Streamlit deployment. Through it I learned how important it is to understand the data first, to scale features where the algorithm needs it, and to compare several models instead of relying on only one. I also learned how a trained model can be saved and turned into a simple application that a normal user can operate.

References

- Kaggle. Mobile Price Classification dataset. <https://www.kaggle.com/datasets/iabhishekofficial/mobile-price-classification>
- Scikit-learn documentation. <https://scikit-learn.org/stable/>
- Streamlit documentation. <https://docs.streamlit.io/>



Mobile Price Range Predictor

Enter mobile specifications to predict its price range.

Mobile Specifications

Battery Power (mAh)	Number of Cores
1200	4
Bluetooth	Primary Camera (MP)
1	10
Clock Speed (GHz)	Pixel Resolution Height
1.50	800
Dual SIM	Pixel Resolution Width
1	1200
Front Camera (MP)	RAM (MB)
5	2000
4G	Screen Height (cm)

1	12
Internal Memory (GB)	Screen Width (cm)
32	7
Mobile Depth (cm)	Talk Time (hours)
0.50	10
Mobile Weight (g)	3G
140	1
	Touch Screen
	1
	WiFi
	1

Predict Price Range

Predicted Price Range: 1 - Medium Cost